XGBoost Coffee Quality Prediction

Comprehensive Analysis with Multiple Modeling Approaches

2025-12-31

Abstract

This report analyzes the Coffee Quality Institute's Arabica coffee dataset using XGBoost regression to predict Cupper Points (expert quality scores). The analysis compares multiple feature sets (Sensory, Extended, Full), evaluates baseline models, and documents both successful approaches and those found less effective (Lambda/Min_Child_Weight with XGBoost providing modest but statistically significant improvements over Linear Regression. regularization, Bayesian Optimization). Key finding: sensory features dominate prediction,

Contents

```r
# CLEAR ENVIRONMENT
rm(list = ls(all.names = TRUE))

# LOAD PACKAGES
library(xgboost)
library(dplyr)
library(tidyr)
library(ggplot2)
library(gridExtra)
library(knitr)
library(stringr)
library(here)
library(shapviz)

# GLOBAL CHUNK OPTIONS
knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE,
  fig.align = "center",
  fig.width = 8,
  fig.height = 5
)

# GGPLOT THEME
theme_set(theme_minimal())

# GLOBAL CONSTANTS
SEED <- 42
K_FOLDS <- 10
```

1 Introduction

This comprehensive report analyzes coffee quality prediction using XGBoost regression. The analysis incorporates multiple modeling approaches tested across several experiments.

Successful Approaches:

- Full model comparison (Sensory, Extended, Full feature sets vs baselines)
- Enhanced evaluation metrics (MAE, MAPE, Adjusted R^2, Max Error)
- Learning Curves for training dynamics visualization
- Exploratory Data Analysis with Correlation Analysis
- Predicted vs Actual diagnostic plots

Approaches Tested but Found Less Effective:

- Lambda (L2) and Min_Child_Weight regularization tuning
- Bayesian Optimization vs Grid Search comparison

1.1 Configuration

```
config <- data.frame(
  Parameter = c("Random Seed", "Cross-Validation Folds", "Train/Test Split",
                "Early Stopping Rounds", "Max Boosting Rounds"),
  Value = c(SEED, K_FOLDS, "80/20", "50", "2000")
)

kable(config, caption = "Analysis Configuration", format = "markdown")
```

Table 1: Analysis Configuration

Parameter	Value
Random Seed	42
Cross-Validation Folds	10
Train/Test Split	80/20
Early Stopping Rounds	50
Max Boosting Rounds	2000

2 Data Loading and Preprocessing

2.1 Load Data

```
coffee_raw <- read.csv(here("arabica_data_cleaned.csv"))
cat("Raw data:", nrow(coffee_raw), "rows,", ncol(coffee_raw), "columns\n")
```

```
## Raw data: 1311 rows, 44 columns
```

2.2 Clean Categorical Variables

```
# Processing Method
coffee_raw$Processing.Method <- ifelse(
  is.na(coffee_raw$Processing.Method) | coffee_raw$Processing.Method == "",
  "Unknown", coffee_raw$Processing.Method
)
coffee_raw$Processing.Method <- factor(coffee_raw$Processing.Method)

# Variety (Top 4 + Other)
variety_counts <- sort(table(coffee_raw$Variety), decreasing = TRUE)
top_4_varieties <- names(variety_counts)[1:4]
coffee_raw$Variety <- ifelse(
  is.na(coffee_raw$Variety) | coffee_raw$Variety == "" |
    !(coffee_raw$Variety %in% top_4_varieties),
  "Other", coffee_raw$Variety
)
coffee_raw$Variety <- factor(coffee_raw$Variety)

# Country (Top 10 + Other)
country_counts <- sort(table(coffee_raw$Country.of.Origin), decreasing = TRUE)
top_10_countries <- names(country_counts)[1:10]
coffee_raw$Country.of.Origin <- ifelse(
  is.na(coffee_raw$Country.of.Origin) |
    !(coffee_raw$Country.of.Origin %in% top_10_countries),
  "Other", coffee_raw$Country.of.Origin
)
coffee_raw$Country.of.Origin <- factor(coffee_raw$Country.of.Origin)

# Color
coffee_raw$Color <- ifelse(
  is.na(coffee_raw$Color) | coffee_raw$Color == "",
  "Unknown", coffee_raw$Color
)
coffee_raw$Color <- factor(coffee_raw$Color)

# In-Country Partner (Top 10 + Other)
partner_counts <- sort(table(coffee_raw$In.Country.Partner), decreasing = TRUE)
top_10_partners <- names(partner_counts)[1:10]
```

```r
coffee_raw$In.Country.Partner <- ifelse(
  is.na(coffee_raw$In.Country.Partner) |
    !(coffee_raw$In.Country.Partner %in% top_10_partners),
  "Other", coffee_raw$In.Country.Partner
)
coffee_raw$In.Country.Partner <- factor(coffee_raw$In.Country.Partner)

# Region (Top 15 + Other)
region_counts <- sort(table(coffee_raw$Region), decreasing = TRUE)
top_15_regions <- names(region_counts)[1:15]
coffee_raw$Region <- ifelse(
  is.na(coffee_raw$Region) | coffee_raw$Region == "" |
    !(coffee_raw$Region %in% top_15_regions),
  "Other", coffee_raw$Region
)
coffee_raw$Region <- factor(coffee_raw$Region)

cat("Categorical variables processed\n")
```

```
## Categorical variables processed
```

```r
cat("  Processing.Method:", nlevels(coffee_raw$Processing.Method), "levels\n")
```

```
##   Processing.Method: 6 levels
```

```r
cat("  Variety:", nlevels(coffee_raw$Variety), "levels\n")
```

```
##   Variety: 4 levels
```

```r
cat("  Country:", nlevels(coffee_raw$Country.of.Origin), "levels\n")
```

```
##   Country: 11 levels
```

```r
cat("  Color:", nlevels(coffee_raw$Color), "levels\n")
```

```
##   Color: 5 levels
```

2.3 Clean Numeric Variables

```r
# Altitude (Cap at 3000m and Impute)
coffee_raw$altitude_mean_meters <- ifelse(
  coffee_raw$altitude_mean_meters > 3000 & !is.na(coffee_raw$altitude_mean_meters),
  3000, coffee_raw$altitude_mean_meters
)

region_medians <- coffee_raw %>%
  filter(!is.na(altitude_mean_meters)) %>%
  group_by(Region) %>%
  summarise(region_alt = median(altitude_mean_meters), .groups = "drop")
```

```r
global_median_alt <- median(coffee_raw$altitude_mean_meters, na.rm = TRUE)

coffee_raw <- coffee_raw %>%
  left_join(region_medians, by = "Region") %>%
  mutate(altitude_mean_meters = ifelse(
    is.na(altitude_mean_meters),
    ifelse(is.na(region_alt), global_median_alt, region_alt),
    altitude_mean_meters
  )) %>%
  select(-region_alt)

# Harvest Year Parsing
parse_harvest_year <- function(val) {
  if (is.na(val) || val == "") return(NA)
  match <- str_extract(as.character(val), "(20\\d{2}|19\\d{2})")
  if (!is.na(match)) return(as.integer(match))
  return(NA)
}

coffee_raw$Harvest.Year.Parsed <- sapply(coffee_raw$Harvest.Year, parse_harvest_year)
median_year <- median(coffee_raw$Harvest.Year.Parsed, na.rm = TRUE)
coffee_raw$Harvest.Year.Parsed <- ifelse(
  is.na(coffee_raw$Harvest.Year.Parsed), median_year, coffee_raw$Harvest.Year.Parsed
)

# Bag Weight Standardization
parse_bag_weight_kg <- function(val) {
  if (is.na(val)) return(NA)
  val <- tolower(as.character(val))
  num <- as.numeric(str_extract(val, "[\\d.]+"))
  if (is.na(num)) return(NA)
  if (grepl("lb", val)) return(num * 0.453592)
  if (grepl("kg", val)) return(num)
  return(ifelse(num > 10, num, NA))
}

coffee_raw$Bag.Weight.KG <- sapply(coffee_raw$Bag.Weight, parse_bag_weight_kg)
median_weight <- median(coffee_raw$Bag.Weight.KG, na.rm = TRUE)
coffee_raw$Bag.Weight.KG <- ifelse(
  is.na(coffee_raw$Bag.Weight.KG), median_weight, coffee_raw$Bag.Weight.KG
)

cat("Numeric variables processed\n")
```

```
## Numeric variables processed
```

```r
cat("  Altitude missing:", sum(is.na(coffee_raw$altitude_mean_meters)), "\n")
```

```
##   Altitude missing: 0
```

```r
cat("  Harvest.Year range:", min(coffee_raw$Harvest.Year.Parsed), "-",
    max(coffee_raw$Harvest.Year.Parsed), "\n")
```

```
##   Harvest.Year range: 2009 - 2018
```

2.4 Remove Leakage Columns

```r
coffee_raw$Total.Cup.Points <- NULL
coffee_raw$Altitude <- NULL
coffee_raw$Species <- NULL
coffee_raw$Harvest.Year <- NULL
coffee_raw$Bag.Weight <- NULL

cat("Leakage columns removed: Total.Cup.Points, Altitude, Species, Harvest.Year, Bag.Weight\n")
```

```
## Leakage columns removed: Total.Cup.Points, Altitude, Species, Harvest.Year, Bag.Weight
```

3 Exploratory Data Analysis

3.1 Feature Descriptions

```
feature_desc <- data.frame(
  Feature = c("Aroma", "Flavor", "Aftertaste", "Acidity", "Body", "Balance",
              "Uniformity", "Clean.Cup", "Sweetness", "Moisture",
              "Category.One.Defects", "Quakers"),
  Description = c("Smell/fragrance of the coffee",
                  "Overall taste profile",
                  "Lingering taste after swallowing",
                  "Bright, tangy quality",
                  "Weight/thickness on the tongue",
                  "How well flavors harmonize",
                  "Consistency across multiple cups",
                  "Absence of off-flavors/defects",
                  "Natural sweetness perception",
                  "Moisture content of beans (%)",
                  "Count of major defects",
                  "Unripe/underdeveloped beans count"),
  Scale = c(rep("0-10", 9), "%", "Count", "Count")
)

kable(feature_desc, caption = "Sensory Feature Descriptions", format = "markdown")
```

Table 2: Sensory Feature Descriptions

Feature	Description	Scale
Aroma	Smell/fragrance of the coffee	0-10
Flavor	Overall taste profile	0-10
Aftertaste	Lingering taste after swallowing	0-10
Acidity	Bright, tangy quality	0-10
Body	Weight/thickness on the tongue	0-10
Balance	How well flavors harmonize	0-10
Uniformity	Consistency across multiple cups	0-10
Clean.Cup	Absence of off-flavors/defects	0-10
Sweetness	Natural sweetness perception	0-10
Moisture	Moisture content of beans (%)	%
Category.One.Defects	Count of major defects	Count
Quakers	Unripe/underdeveloped beans count	Count

3.2 Target Variable Analysis

```
sensory_cols <- c("Aroma", "Flavor", "Aftertaste", "Acidity", "Body",
                  "Balance", "Uniformity", "Clean.Cup", "Sweetness",
                  "Moisture", "Category.One.Defects", "Quakers")
```

```r
target_stats <- data.frame(
  Statistic = c("Mean", "Std Dev", "Median", "Min", "Max", "IQR"),
  Value = c(
    round(mean(coffee_raw$Cupper.Points, na.rm = TRUE), 4),
    round(sd(coffee_raw$Cupper.Points, na.rm = TRUE), 4),
    round(median(coffee_raw$Cupper.Points, na.rm = TRUE), 4),
    min(coffee_raw$Cupper.Points, na.rm = TRUE),
    max(coffee_raw$Cupper.Points, na.rm = TRUE),
    round(IQR(coffee_raw$Cupper.Points, na.rm = TRUE), 4)
  )
)
kable(target_stats, caption = "Target Variable (Cupper.Points) Summary", format = "markdown")
```

Table 3: Target Variable (Cupper.Points) Summary

Statistic	Value
Mean	7.4979
Std Dev	0.4746
Median	7.5000
Min	0.0000
Max	10.0000
IQR	0.5000

```r
ggplot(coffee_raw, aes(x = Cupper.Points)) +
  geom_histogram(bins = 30, fill = "#2E86AB", color = "white", alpha = 0.8) +
  geom_vline(xintercept = mean(coffee_raw$Cupper.Points, na.rm = TRUE),
             color = "red", linetype = "dashed", linewidth = 1) +
  labs(x = "Cupper Points", y = "Count")
```

3.3 Correlation Analysis

```r
numeric_cols <- coffee_raw[, c(sensory_cols, "Cupper.Points")]
numeric_cols <- numeric_cols[complete.cases(numeric_cols), ]

correlations <- cor(numeric_cols)
target_corr <- correlations["Cupper.Points", ]
target_corr_sorted <- sort(target_corr[names(target_corr) != "Cupper.Points"],
                           decreasing = TRUE)

corr_df <- data.frame(
  Feature = names(target_corr_sorted),
  Correlation = round(as.numeric(target_corr_sorted), 4)
)
kable(corr_df, caption = "Feature Correlations with Cupper.Points", format = "markdown")
```

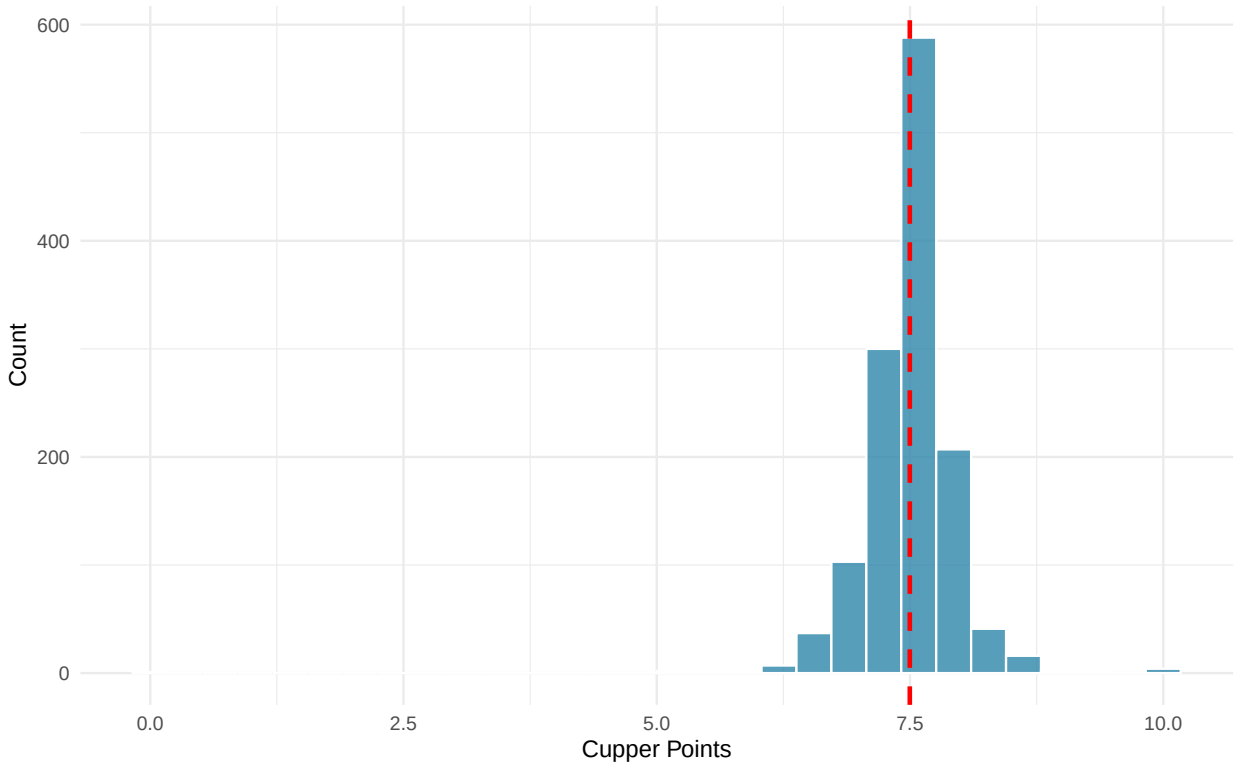


Figure 1: Distribution of Cupper Points. Red dashed line indicates mean.

Table 4: Feature Correlations with Cupper.Points

Feature	Correlation
Flavor	0.7971
Aftertaste	0.7878
Balance	0.7415
Acidity	0.7004
Aroma	0.6934
Body	0.6711
Uniformity	0.3585
Clean.Cup	0.3570
Sweetness	0.3001
Quakers	0.0119
Category.One.Defects	-0.0527
Moisture	-0.1792

```
ggplot(corr_df, aes(x = reorder(Feature, Correlation), y = Correlation,
                    fill = Correlation > 0)) +
  geom_col(alpha = 0.8) +
  coord_flip() +
  scale_fill_manual(values = c("TRUE" = "#2E86AB", "FALSE" = "#E63946"),
```

```
                    guide = "none") +
  labs(x = "Feature", y = "Correlation") +
  geom_hline(yintercept = 0, linetype = "dashed")
```

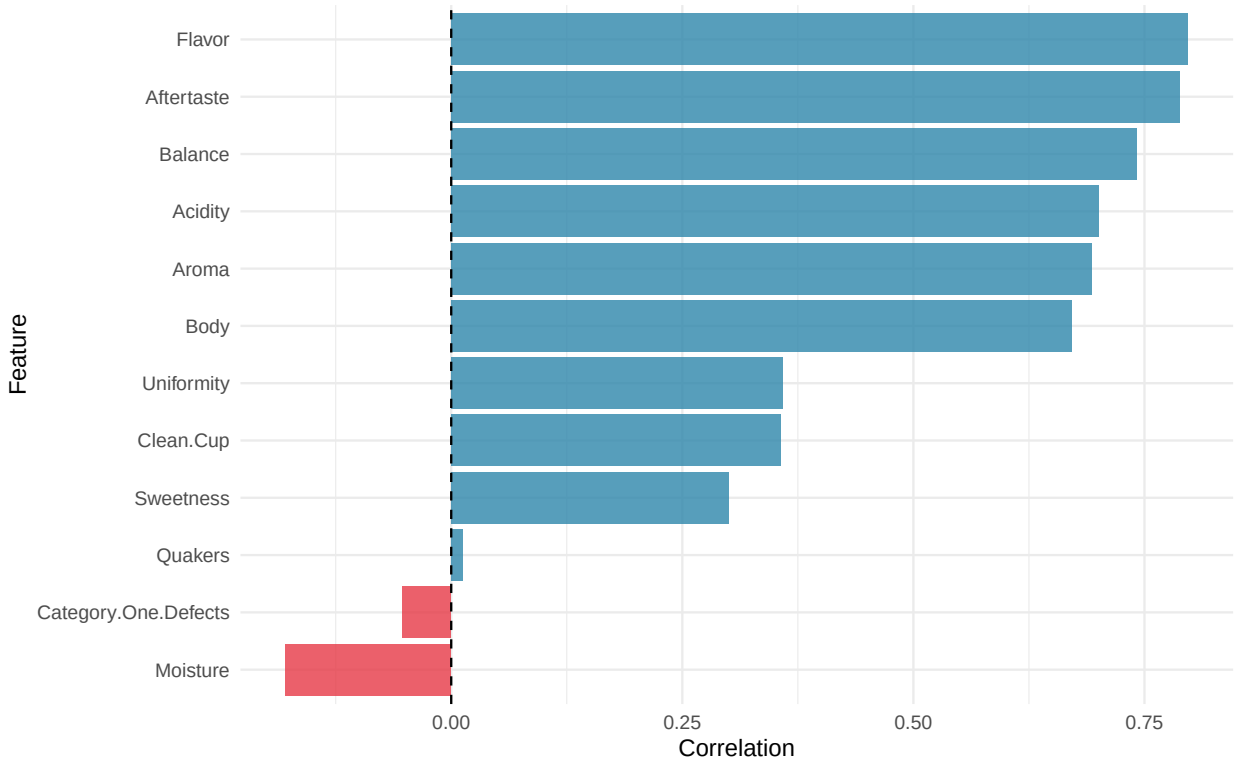


Figure 2: Feature correlations with Cupper Points.

```
cor_matrix <- cor(numeric_cols)
cor_melted <- as.data.frame(as.table(cor_matrix))
names(cor_melted) <- c("Var1", "Var2", "Correlation")

ggplot(cor_melted, aes(x = Var1, y = Var2, fill = Correlation)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low = "#E63946", mid = "white", high = "#2E86AB",
                       midpoint = 0, limits = c(-1, 1)) +
  geom_text(aes(label = round(Correlation, 2)), size = 2.5) +
  labs(x = "", y = "") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

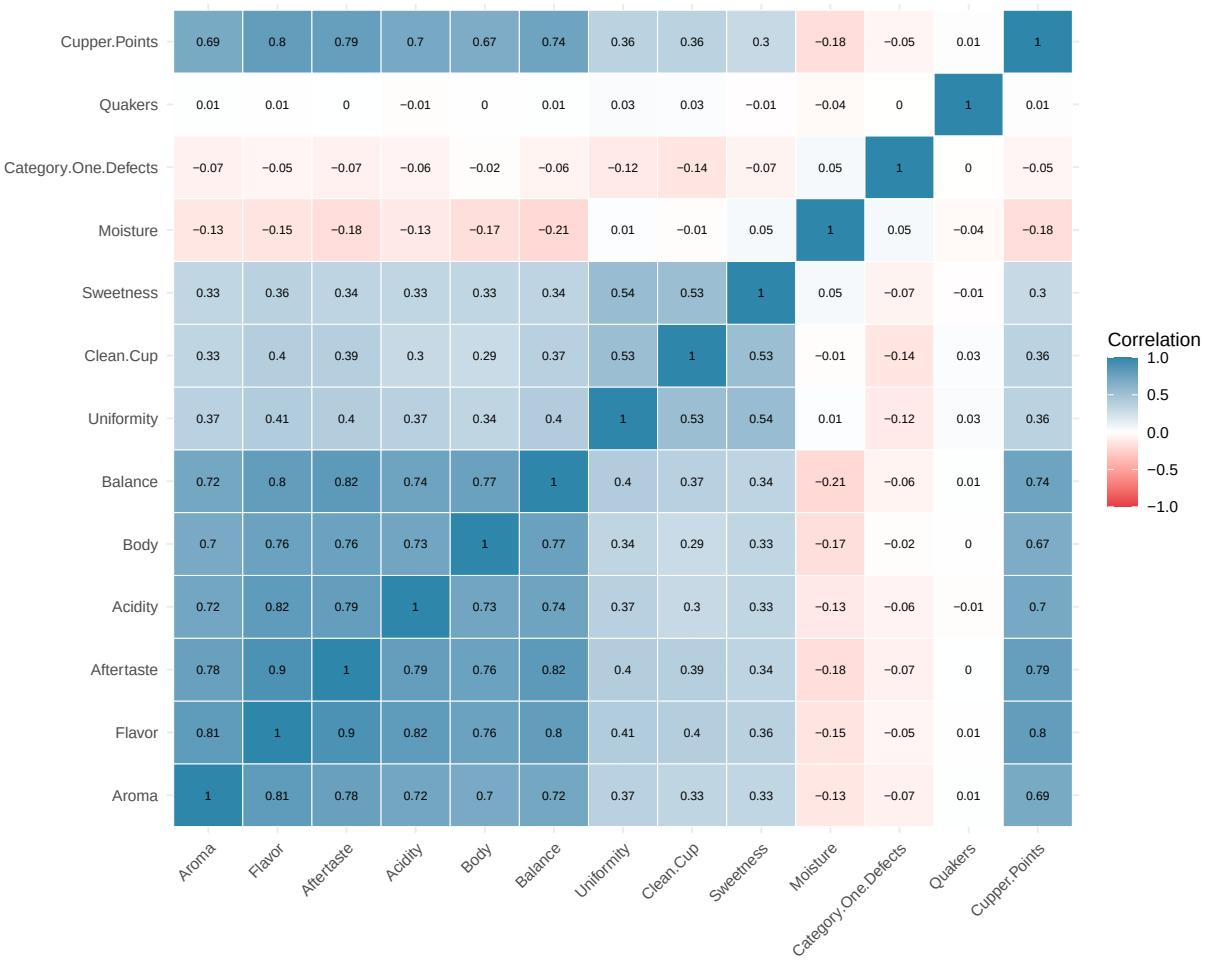


Figure 3: Correlation heatmap of sensory features and target variable.

4 Feature Engineering

4.1 Feature Set Definitions

```
sensory_cols <- c("Aroma", "Flavor", "Aftertaste", "Acidity", "Body",
                  "Balance", "Uniformity", "Clean.Cup", "Sweetness",
                  "Moisture", "Category.One.Defects", "Quakers")

extended_cols <- c(sensory_cols,
                   "Processing.Method", "Variety",
                   "altitude_mean_meters", "Category.Two.Defects")

full_cols <- c(extended_cols,
               "Country.of.Origin", "Color", "Region",
               "Harvest.Year.Parsed", "Bag.Weight.KG",
               "Number.of.Bags", "In.Country.Partner")
```

```
feature_sets <- data.frame(
  Feature.Set = c("Sensory", "Extended", "Full"),
  N.Features = c(length(sensory_cols), length(extended_cols), length(full_cols)),
  Description = c("Core sensory quality indicators only",
                  "Sensory + key metadata (processing, variety, altitude)",
                  "All available features including origin and logistics")
)
kable(feature_sets, caption = "Feature Set Definitions", format = "markdown")
```

Table 5: Feature Set Definitions

Feature.Set	N.Features	Description
Sensory	12	Core sensory quality indicators only
Extended	16	Sensory + key metadata (processing, variety, altitude)
Full	23	All available features including origin and logistics

4.2 Prepare Clean Dataset

```
coffee_clean <- coffee_raw[, c(full_cols, "Cupper.Points")]
coffee_clean <- na.omit(coffee_clean)
cat("Clean dataset:", nrow(coffee_clean), "rows\n")
```

```
## Clean dataset: 1310 rows
```

5 Train/Test Split

```r
set.seed(SEED)

n <- nrow(coffee_clean)
train_idx <- sample(1:n, floor(0.8 * n))
test_idx <- setdiff(1:n, train_idx)

y <- coffee_clean$Cupper.Points
y_train <- y[train_idx]
y_test <- y[test_idx]

# Sensory features
X_sensory <- as.matrix(coffee_clean[, sensory_cols])
X_sensory_train <- X_sensory[train_idx, ]
X_sensory_test <- X_sensory[test_idx, ]

# Extended features (one-hot encoded)
X_extended <- model.matrix(~ . - 1, data = coffee_clean[, extended_cols])
X_extended_train <- X_extended[train_idx, ]
X_extended_test <- X_extended[test_idx, ]

# Full features (one-hot encoded)
X_full <- model.matrix(~ . - 1, data = coffee_clean[, full_cols])
X_full_train <- X_full[train_idx, ]
X_full_test <- X_full[test_idx, ]

train_df <- coffee_clean[train_idx, ]
test_df <- coffee_clean[test_idx, ]

# CV folds
set.seed(SEED + 42)
fold_ids <- sample(rep(1:K_FOLDS, length.out = n))
```

```r
split_summary <- data.frame(
  Set = c("Training", "Test", "Total"),
  Observations = c(length(train_idx), length(test_idx), n),
  Percentage = c("80%", "20%", "100%")
)
kable(split_summary, caption = "Train/Test Split Summary", format = "markdown")
```

Table 6: Train/Test Split Summary

Set	Observations	Percentage
Training	1048	80%
Test	262	20%
Total	1310	100%

6 Evaluation Metrics

```r
get_metrics <- function(pred, obs) {
  residuals <- obs - pred
  rmse <- sqrt(mean(residuals^2))
  ss_res <- sum(residuals^2)
  ss_tot <- sum((obs - mean(obs))^2)
  r2 <- 1 - (ss_res / ss_tot)
  mae <- mean(abs(residuals))
  return(list(RMSE = rmse, R2 = r2, MAE = mae))
}

get_enhanced_metrics <- function(pred, obs, n_features = 12) {
  residuals <- obs - pred
  n <- length(obs)

  rmse <- sqrt(mean(residuals^2))
  mae <- mean(abs(residuals))
  median_ae <- median(abs(residuals))
  max_error <- max(abs(residuals))
  mape <- mean(abs(residuals / (obs + 1e-10))) * 100

  ss_res <- sum(residuals^2)
  ss_tot <- sum((obs - mean(obs))^2)
  r2 <- 1 - (ss_res / ss_tot)
  adj_r2 <- 1 - (1 - r2) * (n - 1) / (n - n_features - 1)

  return(list(
    RMSE = rmse, MAE = mae, MAPE = mape, R2 = r2,
    Adjusted_R2 = adj_r2, Median_AE = median_ae, Max_Error = max_error
  ))
}

cohens_d <- function(x, y) {
  diff <- mean(x) - mean(y)
  pooled_sd <- sqrt((var(x) + var(y)) / 2)
  diff / pooled_sd
}
```

7 Baseline Models

7.1 Mean Predictor

```
mean_train <- mean(y_train)
preds_mean <- rep(mean_train, length(y_test))
metrics_mean <- get_metrics(preds_mean, y_test)
metrics_mean_enh <- get_enhanced_metrics(preds_mean, y_test, 0)

cat("Training Mean:", round(mean_train, 4), "\n")
```

```
## Training Mean: 7.4939
```

```
cat("Test RMSE:", round(metrics_mean$RMSE, 4), "\n")
```

```
## Test RMSE: 0.44
```

```
cat("Test R²:", round(metrics_mean$R2, 4), "\n")
```

```
## Test R²: -0.002
```

7.2 Linear Regression

```
model_lm <- lm(Cupper.Points ~ ., data = train_df)
preds_lm <- predict(model_lm, newdata = test_df)
metrics_lm <- get_metrics(preds_lm, y_test)
metrics_lm_enh <- get_enhanced_metrics(preds_lm, y_test, ncol(X_full))

cat("Linear Regression Test RMSE:", round(metrics_lm$RMSE, 4), "\n")
```

```
## Linear Regression Test RMSE: 0.3035
```

```
cat("Linear Regression Test R²:", round(metrics_lm$R2, 4), "\n")
```

```
## Linear Regression Test R²: 0.5234
```

```
lm_coefs <- summary(model_lm)$coefficients
lm_coefs_df <- data.frame(
  Feature = rownames(lm_coefs),
  Estimate = round(lm_coefs[, 1], 4),
  Std_Error = round(lm_coefs[, 2], 4),
  P_Value = format(lm_coefs[, 4], digits = 3)
)
kable(head(lm_coefs_df[order(abs(lm_coefs_df$Estimate), decreasing = TRUE), ], 10),
      caption = "Top 10 Linear Regression Coefficients",
      format = "markdown", row.names = FALSE)
```

Table 7: Top 10 Linear Regression Coefficients

Feature	Estimate	Std_Error	P_Value
(Intercept)	-22.7192	12.6256	7.23e-02
Flavor	0.3815	0.0529	1.10e-12
Processing.MethodOther	-0.2629	0.0634	3.63e-05
Aftertaste	0.2403	0.0523	5.02e-06
Balance	0.2220	0.0399	3.41e-08
Processing.MethodUnknown	0.1827	0.0372	1.04e-06
ColorUnknown	-0.1741	0.0425	4.61e-05
In.Country.PartnerBlossom Valley International	0.1611	0.0852	5.91e-02
Moisture	-0.1431	0.1981	4.70e-01
ColorGreen	-0.1413	0.0345	4.60e-05

8 XGBoost XAI

8.1 ALE

8.1.1 XGBoost Load Model

```
model_xgb_f <- xgb.load("models/model_xgb_f.xgb")

dtest_f <- xgb.DMatrix(data = X_full_test, label = y_test)
preds_xgb_f <- predict(model_xgb_f, dtest_f)
```

9 Explainable AI: PFI and PDP Analysis

This section applies model-agnostic XAI techniques to interpret the best XGBoost Full model. We focus on:

- **Permutation Feature Importance (PFI)**: Which features matter most?
- **Partial Dependence Plots (PDP)**: How do individual features affect predictions?
- **Accumulated Local Effects (ALE)**: Similar to PDP but more robust to multicollinearities.
- **Shapley Additive Explanations (SHAP)**:
- **Microscopic Explanations**: Why did specific predictions occur?

9.1 Setup: Load DALEX and Create Explainer

```
# Install DALEX if not available
if (!require("DALEX")) install.packages("DALEX")
library(DALEX)

# Custom prediction function for XGBoost
# DALEX requires a function that takes (model, newdata) and returns numeric predictions
predict_xgb <- function(model, newdata) {
  # Convert to matrix if it's a data frame
  if (is.data.frame(newdata)) {
    newdata <- as.matrix(newdata)
  }
  dmatrix <- xgb.DMatrix(data = newdata)
  predict(model, dmatrix)
}

# Create DALEX explainer for XGBoost Full model
# The explainer acts as a unified interface for all XAI methods
explainer_xgb_full <- DALEX::explain(
  model = model_xgb_f,
  data = X_full_train,           # Training features (matrix)
  y = y_train,                   # Training target values
  predict_function = predict_xgb,
  label = "XGBoost Full",
  verbose = FALSE
```

```
)

cat("Explainer created successfully!\n")
```

```
## Explainer created successfully!
```

```
cat("  Model: XGBoost Full\n")
```

```
##   Model: XGBoost Full
```

```
cat("  Features:", ncol(X_full_train), "\n")
```

```
##   Features: 64
```

```
cat("  Observations:", nrow(X_full_train), "\n")
```

```
##   Observations: 1048
```

9.2 Permutation Feature Importance (PFI)

9.2.1 Concept

PFI measures how much the model's performance degrades when a feature's relationship with the target is broken by randomly shuffling its values. The algorithm:

1. Calculate baseline error: $e_{orig} = L(y, f(X))$
2. For each feature S: shuffle its values to create $X_{S,perm}$
3. Calculate new error: $e_{S,perm} = L(y, f(X_{S,perm}))$
4. Feature importance: $FI_S = e_{S,perm} - e_{orig}$

Higher importance = larger performance drop = more important feature.

9.2.2 Calculate PFI

```
# Custom loss function: Mean Absolute Error (more interpretable for Cupper Points)
loss_mae <- function(observed, predicted) {
  mean(abs(observed - predicted))
}

# Calculate PFI using MAE loss
# B = number of permutation repetitions for stability
set.seed(SEED)
pfi_xgb_full <- model_parts(
  explainer = explainer_xgb_full,
  loss_function = loss_mae,
  type = "difference",     # Report change in MAE (not ratio)
  B = 10                    # 10 permutation repetitions
)

cat("PFI calculation complete.\n")
```

```
## PFI calculation complete.
```

9.2.3 PFI Visualization

```
# Plot PFI - shows top features by importance
plot(pfi_xgb_full, max_vars = 20) +
  ggtitle("Permutation Feature Importance (Top 20 Features)") +
  xlab("MAE Increase When Feature is Permuted") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"))
```

```
## NULL
```

9.2.4 PFI Results Table

```
# Extract and format PFI results
pfi_results <- pfi_xgb_full %>%
```

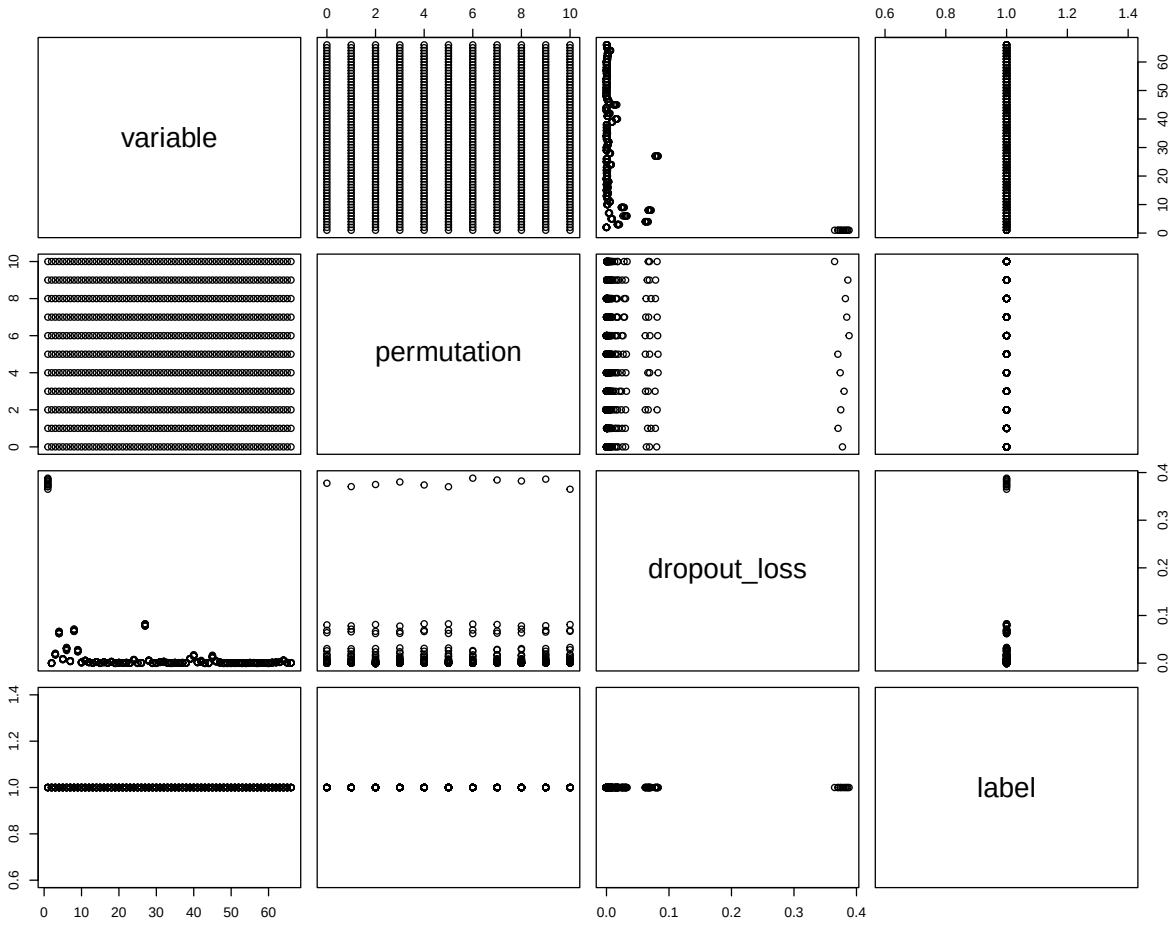


Figure 4: Permutation Feature Importance for XGBoost Full model. Longer bars indicate features whose shuffling causes larger MAE increases, meaning they are more important for prediction accuracy.

```
  filter(variable != "_baseline_" & variable != "_full_model_") %>%
  group_by(variable) %>%
  summarise(
    Mean_Dropout_Loss = mean(dropout_loss),
    SD = sd(dropout_loss),
    .groups = "drop"
  ) %>%
  arrange(desc(Mean_Dropout_Loss)) %>%
  mutate(
    Importance = Mean_Dropout_Loss - min(pfi_xgb_full$dropout_loss[pfi_xgb_full$variable == "_
    Rank = row_number()
  )

# Display top 15
kable(head(pfi_results[, c("Rank", "variable", "Importance", "SD")], 15),
      col.names = c("Rank", "Feature", "MAE Increase", "SD"),
      caption = "Top 15 Features by Permutation Feature Importance",
      format = "markdown", digits = 4)
```

Table 8: Top 15 Features by Permutation Feature Importance

Rank	Feature	MAE Increase	SD
1	Flavor	0.0811	0.0018
2	Balance	0.0697	0.0015
3	Aftertaste	0.0647	0.0018
4	Aroma	0.0314	0.0017
5	Body	0.0266	0.0016
6	Acidity	0.0194	0.0010
7	Number.of.Bags	0.0166	0.0009
8	Processing.MethodUnknown	0.0144	0.0015
9	Moisture	0.0097	0.0006
10	altitude_mean_meters	0.0094	0.0006
11	Country.of.OriginTaiwan	0.0074	0.0007
12	VarietyCaturra	0.0064	0.0006
13	Category.Two.Defects	0.0062	0.0004
14	Harvest.Year.Parsed	0.0062	0.0007
15	Bag.Weight.KG	0.0051	0.0005

9.2.5 PFI Interpretation

```
# Identify top 5 features
top5_features <- head(pfi_results$variable, 5)
top5_importance <- head(pfi_results$Importance, 5)

cat("KEY PFI FINDINGS:\n")
```

```
## KEY PFI FINDINGS:
```

```r
cat("================\n\n")
```

```
## ================
```

```r
cat("Top 5 Most Important Features (by MAE increase when permuted):\n\n")
```

```
## Top 5 Most Important Features (by MAE increase when permuted):
```

```r
for (i in 1:5) {
  cat(sprintf("  %d. %s: +%.4f MAE\n", i, top5_features[i], top5_importance[i]))
}
```

```
##   1. Flavor: +0.0811 MAE
##   2. Balance: +0.0697 MAE
##   3. Aftertaste: +0.0647 MAE
##   4. Aroma: +0.0314 MAE
##   5. Body: +0.0266 MAE
```

```r
cat("\n")
```

```r
cat("Interpretation: Shuffling these features causes the largest prediction errors,\n")
```

```
## Interpretation: Shuffling these features causes the largest prediction errors,
```

```r
cat("indicating the model relies heavily on them for accurate predictions.\n")
```

```
## indicating the model relies heavily on them for accurate predictions.
```

9.3 Partial Dependence Plots (PDP)

9.3.1 Concept

PDP shows the **marginal effect** of a feature on predictions, averaging over all other features:

$$\hat{f}_S(s^*) = \frac{1}{n}\sum_{i=1}^{n} f(S = s^*, \mathbf{C}_i)$$

Where: - s^* = a specific value of feature S - C_i = values of all other features for observation i - The sum averages predictions across all observations with S fixed at s^*

9.3.2 Calculate PDPs for Top Sensory Features

```
# Identify top sensory features from PFI (these are most interpretable)
sensory_features_in_model <- intersect(
  sensory_cols,
  colnames(X_full_train)
)

# Get top 4 sensory features by PFI importance
top_sensory_pfi <- pfi_results %>%
  filter(variable %in% sensory_features_in_model) %>%
  head(4) %>%
  pull(variable)

cat("Calculating PDPs for top sensory features:", paste(top_sensory_pfi, collapse = ", "), "\n"
```

```
## Calculating PDPs for top sensory features: Flavor, Balance, Aftertaste, Aroma
```

```
# Calculate PDPs for each top feature
pdp_list <- list()
for (feat in top_sensory_pfi) {
  pdp_list[[feat]] <- model_profile(
    explainer = explainer_xgb_full,
    variables = feat,
    type = "partial",
    N = 300  # Use 300 observations for speed (set higher for smoother curves)
  )
}

cat("PDP calculation complete.\n")
```

```
## PDP calculation complete.
```

9.3.3 Combined PDP Plot

```
# Combine all PDPs for comparison
all_pdps <- model_profile(
```

```
  explainer = explainer_xgb_full,
  variables = top_sensory_pfi,
  type = "partial",
  N = 300
)

plot(all_pdps) +
  ggtitle("Combined Partial Dependence Plots") +
  ylab("Average Predicted Cupper Points") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        legend.position = "bottom")
```

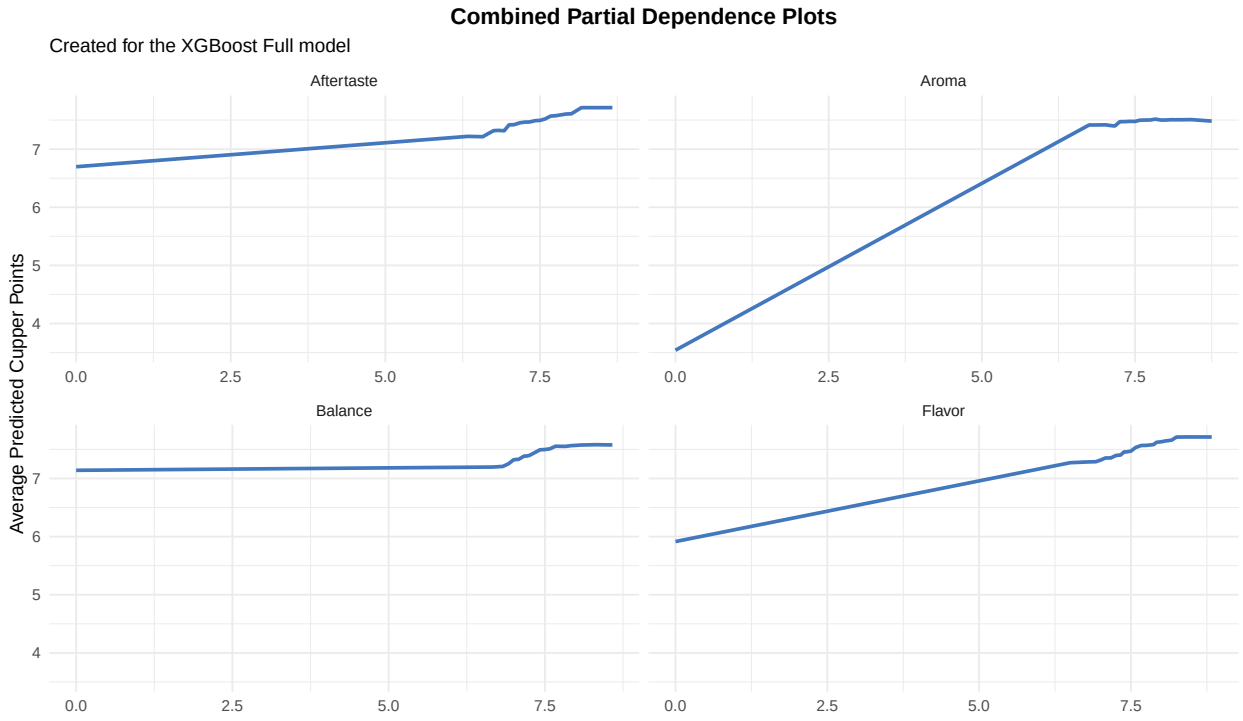


Figure 5: Combined PDP showing all top sensory features on the same scale for direct comparison of effect magnitudes.

9.3.4 PDP for Feature Interactions (2D)

```
# 2D PDP for top 2 features (if both are sensory)
if (length(top_sensory_pfi) >= 2) {
  pdp_2d <- model_profile(
    explainer = explainer_xgb_full,
    variables = top_sensory_pfi[1:2],
    type = "partial",
    N = 100  # Fewer observations for 2D (computational)
```

```
  )

  # Extract the data for custom 2D visualization
  cat("2D PDP calculated for:", top_sensory_pfi[1], "and", top_sensory_pfi[2], "\n")
  cat("(2D visualization requires additional packages - showing individual PDPs above)\n")
}
```

```
## 2D PDP calculated for: Flavor and Balance
## (2D visualization requires additional packages - showing individual PDPs above)
```

9.3.5 PDP Interpretation

```
cat("KEY PDP FINDINGS:\n")
```

```
## KEY PDP FINDINGS:
```

```
cat("================\n\n")
```

```
## ================
```

```
for (feat in top_sensory_pfi) {
  pdp_data <- pdp_list[[feat]]$agr_profiles
  min_pred <- min(pdp_data$`_yhat_`)
  max_pred <- max(pdp_data$`_yhat_`)
  range_effect <- max_pred - min_pred

  # Find the x values at min and max predictions
  x_at_min <- pdp_data$`_x_`[which.min(pdp_data$`_yhat_`)]
  x_at_max <- pdp_data$`_x_`[which.max(pdp_data$`_yhat_`)]

  cat(sprintf("%s:\n", feat))
  cat(sprintf("  - Effect range: %.4f Cupper Points\n", range_effect))
  cat(sprintf("  - Lowest avg prediction (%.3f) at %s = %.2f\n", min_pred, feat, x_at_min))
  cat(sprintf("  - Highest avg prediction (%.3f) at %s = %.2f\n", max_pred, feat, x_at_max))
  cat("\n")
}
```

```
## Flavor:
##   - Effect range: 1.8004 Cupper Points
##   - Lowest avg prediction (5.903) at Flavor = 0.00
##   - Highest avg prediction (7.704) at Flavor = 8.42
##
## Balance:
##   - Effect range: 0.4302 Cupper Points
##   - Lowest avg prediction (7.166) at Balance = 0.00
##   - Highest avg prediction (7.596) at Balance = 8.33
##
## Aftertaste:
##   - Effect range: 1.0101 Cupper Points
```

```
##   - Lowest avg prediction (6.705) at Aftertaste = 0.00
##   - Highest avg prediction (7.715) at Aftertaste = 8.67
##
## Aroma:
##   - Effect range: 3.9898 Cupper Points
##   - Lowest avg prediction (3.524) at Aroma = 0.00
##   - Highest avg prediction (7.514) at Aroma = 7.83
```

```
cat("Interpretation:\n")
```

```
## Interpretation:
```

```
cat("Features with larger effect ranges have stronger marginal effects on predictions.\n")
```

```
## Features with larger effect ranges have stronger marginal effects on predictions.
```

```
cat("The PDP curves show the 'shape' of each relationship (linear, monotonic, complex).\n")
```

```
## The PDP curves show the 'shape' of each relationship (linear, monotonic, complex).
```

9.3.6 Feature Importance from PDP (Variance-based)

```
# Calculate importance from PDP as variance of PDP values
pdp_variance_importance <- data.frame(
  Feature = character(),
  PDP_Variance = numeric(),
  PDP_Range = numeric(),
  stringsAsFactors = FALSE
)

for (feat in top_sensory_pfi) {
  pdp_data <- pdp_list[[feat]]$agr_profiles
  pdp_variance_importance <- rbind(pdp_variance_importance, data.frame(
    Feature = feat,
    PDP_Variance = var(pdp_data$`_yhat_`),
    PDP_Range = max(pdp_data$`_yhat_`) - min(pdp_data$`_yhat_`)
  ))
}

pdp_variance_importance <- pdp_variance_importance %>%
  arrange(desc(PDP_Variance))

kable(pdp_variance_importance,
      col.names = c("Feature", "PDP Variance", "PDP Range"),
      caption = "Feature Importance Derived from PDP (Higher Variance = Stronger Effect)",
      format = "markdown", digits = 4)
```

Table 9: Feature Importance Derived from PDP (Higher Variance = Stronger Effect)

Feature	PDP Variance	PDP Range
Aroma	0.6499	3.9898
Flavor	0.1221	1.8004
Aftertaste	0.0442	1.0101
Balance	0.0190	0.4302

9.4 Microscopic Explanations (Individual Predictions)

9.4.1 Concept

While PFI and PDP provide **global** insights, microscopic explanations show why the model made a **specific prediction** for individual observations.

We use DALEX's **Break Down** method, which decomposes a prediction into feature contributions (note: this is NOT SHAP, but a complementary approach).

9.4.2 Select Observations for Explanation

```
# Select interesting observations from test set
# 1. Best prediction (closest to actual)
# 2. Worst prediction (furthest from actual)
# 3. Median prediction

residuals_test <- y_test - preds_xgb_f
abs_residuals <- abs(residuals_test)

idx_best <- which.min(abs_residuals)
idx_worst <- which.max(abs_residuals)
idx_median <- which.min(abs(abs_residuals - median(abs_residuals)))

selected_obs <- data.frame(
  Type = c("Best Prediction", "Worst Prediction", "Median Prediction"),
  Test_Index = c(idx_best, idx_worst, idx_median),
  Actual = round(y_test[c(idx_best, idx_worst, idx_median)], 3),
  Predicted = round(preds_xgb_f[c(idx_best, idx_worst, idx_median)], 3),
  Residual = round(residuals_test[c(idx_best, idx_worst, idx_median)], 3)
)

kable(selected_obs, caption = "Selected Observations for Microscopic Analysis", format = "mark
```

Table 10: Selected Observations for Microscopic Analysis

Type	Test_Index	Actual	Predicted	Residual
Best Prediction	212	7.25	7.250	0.000
Worst Prediction	202	5.25	7.835	-2.585
Median Prediction	160	7.50	7.397	0.103

9.4.3 Break Down Explanations

```
# Prepare test observations for explanation
new_obs_best <- X_full_test[idx_best, , drop = FALSE]
new_obs_worst <- X_full_test[idx_worst, , drop = FALSE]
new_obs_median <- X_full_test[idx_median, , drop = FALSE]
```

```r
# Calculate Break Down for each observation
bd_best <- predict_parts(
  explainer = explainer_xgb_full,
  new_observation = new_obs_best,
  type = "break_down"  # NOT shap - uses sequential contribution
)

bd_worst <- predict_parts(
  explainer = explainer_xgb_full,
  new_observation = new_obs_worst,
  type = "break_down"
)

bd_median <- predict_parts(
  explainer = explainer_xgb_full,
  new_observation = new_obs_median,
  type = "break_down"
)

# Plot all three
p_best <- plot(bd_best, max_features = 10) +
  ggtitle(sprintf("Best Prediction (Actual: %.2f, Pred: %.2f)",
                  y_test[idx_best], preds_xgb_f[idx_best])) +
  theme(plot.title = element_text(size = 10))

p_worst <- plot(bd_worst, max_features = 10) +
  ggtitle(sprintf("Worst Prediction (Actual: %.2f, Pred: %.2f)",
                  y_test[idx_worst], preds_xgb_f[idx_worst])) +
  theme(plot.title = element_text(size = 10))

p_median <- plot(bd_median, max_features = 10) +
  ggtitle(sprintf("Median Prediction (Actual: %.2f, Pred: %.2f)",
                  y_test[idx_median], preds_xgb_f[idx_median])) +
  theme(plot.title = element_text(size = 10))

grid.arrange(p_best, p_median, p_worst, ncol = 1)
```

9.4.4 Microscopic Explanation Tables

```r
# Extract top contributions for each observation
extract_top_contributions <- function(bd_result, n = 8) {
  bd_df <- as.data.frame(bd_result)
  bd_df <- bd_df %>%
    filter(variable_name != "intercept" & variable_name != "prediction") %>%
    arrange(desc(abs(contribution))) %>%
    head(n) %>%
    select(variable_name, variable_value, contribution) %>%
```

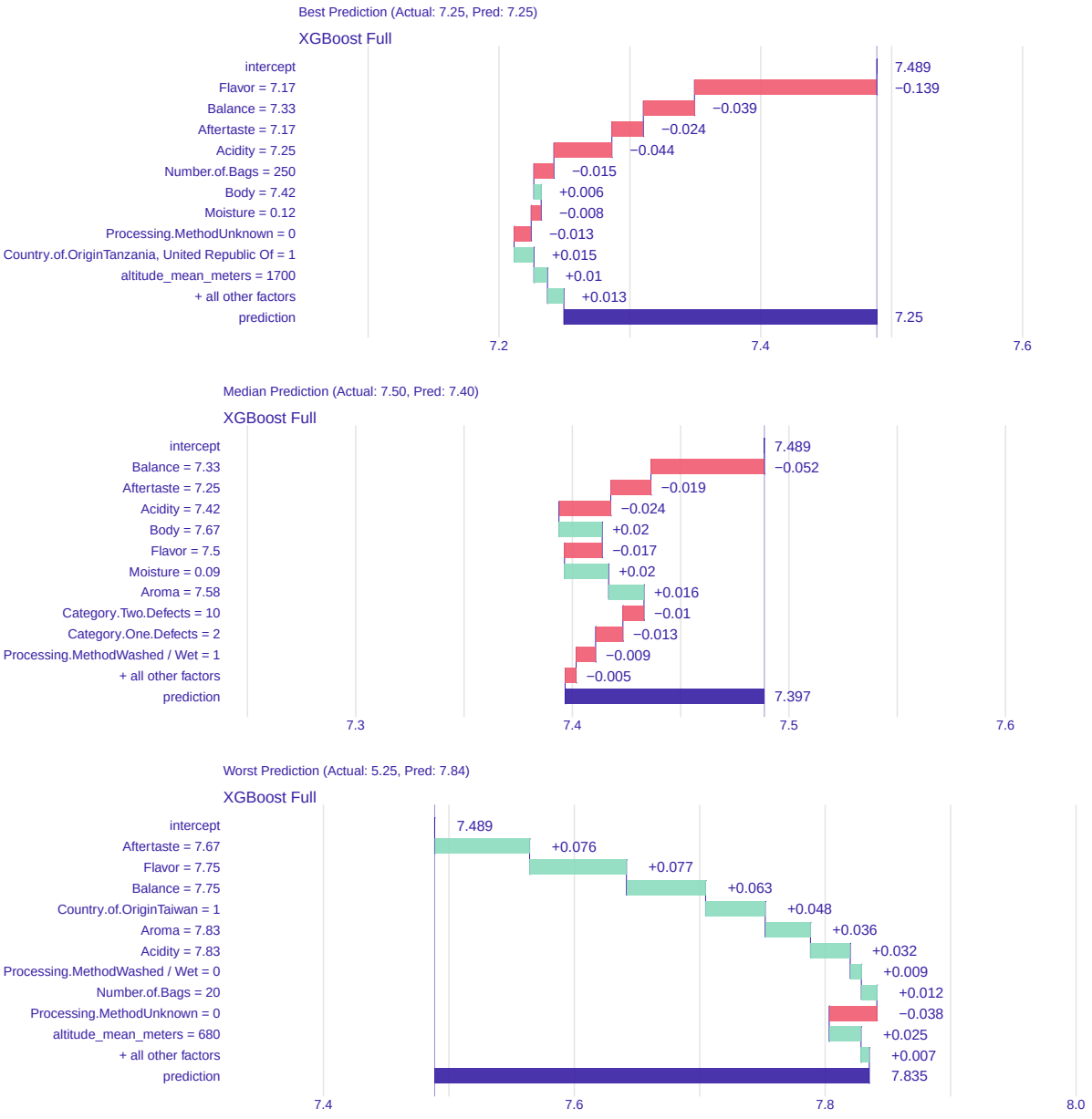


Figure 6: Break Down plots showing how each feature contributes to individual predictions. Green bars push predictions up; red bars push predictions down.

```
    mutate(
      contribution = round(contribution, 4),
      Direction = ifelse(contribution > 0, "↑ Positive", "↓ Negative")
    )
  return(bd_df)
}

cat("Best Prediction - Top Feature Contributions:\n")
```

```
## Best Prediction - Top Feature Contributions:
```

```
kable(extract_top_contributions(bd_best),
      col.names = c("Feature", "Value", "Contribution", "Direction"),
      format = "markdown")
```

Feature	Value	Contribution	Direction
		7.2496	↑ Positive
Flavor	7.17	-0.1393	↓ Negative
Acidity	7.25	-0.0443	↓ Negative
Balance	7.33	-0.0391	↓ Negative
Aftertaste	7.17	-0.0240	↓ Negative
Number.of.Bags	250	-0.0153	↓ Negative
Country.of.OriginTanzania, United Republic Of	1	0.0151	↑ Positive
Processing.MethodUnknown	0	-0.0130	↓ Negative

```
cat("\nWorst Prediction - Top Feature Contributions:\n")
```

```
##
## Worst Prediction - Top Feature Contributions:
```

```
kable(extract_top_contributions(bd_worst),
      col.names = c("Feature", "Value", "Contribution", "Direction"),
      format = "markdown")
```

Feature	Value	Contribution	Direction
		7.8352	↑ Positive
Flavor	7.75	0.0772	↑ Positive
Aftertaste	7.67	0.0757	↑ Positive
Balance	7.75	0.0631	↑ Positive
Country.of.OriginTaiwan	1	0.0475	↑ Positive
Processing.MethodUnknown	0	-0.0380	↓ Negative
Aroma	7.83	0.0361	↑ Positive
Acidity	7.83	0.0317	↑ Positive

9.5 Summary: XAI Insights for XGBoost Full Model

9.5.1 Key Findings from PFI

```
cat("PERMUTATION FEATURE IMPORTANCE SUMMARY\n")
```

```
## PERMUTATION FEATURE IMPORTANCE SUMMARY
```

```
cat("======================================\n\n")
```

```
## ======================================
```

```
# Top 5 features
cat("Most Important Features (by MAE increase when shuffled):\n")
```

```
## Most Important Features (by MAE increase when shuffled):
```

```
for (i in 1:min(5, nrow(pfi_results))) {
  cat(sprintf("  %d. %s (+%.4f MAE)\n", i, pfi_results$variable[i], pfi_results$Importance[i]))
}
```

```
##   1. Flavor (+0.0811 MAE)
##   2. Balance (+0.0697 MAE)
##   3. Aftertaste (+0.0647 MAE)
##   4. Aroma (+0.0314 MAE)
##   5. Body (+0.0266 MAE)
```

```
# Sensory vs non-sensory
sensory_in_top10 <- sum(head(pfi_results$variable, 10) %in% sensory_cols)
cat(sprintf("\nSensory features in top 10: %d/10\n", sensory_in_top10))
```

```
##
## Sensory features in top 10: 7/10
```

```
cat("This confirms that expert taste evaluation drives predictions.\n")
```

```
## This confirms that expert taste evaluation drives predictions.
```

9.5.2 Key Findings from PDP

```
cat("\nPARTIAL DEPENDENCE PLOT SUMMARY\n")
```

```
##
## PARTIAL DEPENDENCE PLOT SUMMARY
```

```
cat("================================\n\n")
```

```
## ================================
```

```
cat("Feature Effect Magnitudes:\n")
```

```
## Feature Effect Magnitudes:
```

```
for (i in 1:nrow(pdp_variance_importance)) {
  cat(sprintf("  %s: %.4f point range\n",
```

```
              pdp_variance_importance$Feature[i],
              pdp_variance_importance$PDP_Range[i]))
}
```

```
##   Aroma: 3.9898 point range
##   Flavor: 1.8004 point range
##   Aftertaste: 1.0101 point range
##   Balance: 0.4302 point range
```

```
cat("\nRelationship Shapes:\n")
```

```
##
## Relationship Shapes:
```

```
cat("  - Most sensory features show positive monotonic relationships\n")
```

```
##   - Most sensory features show positive monotonic relationships
```

```
cat("  - Higher scores → higher predicted Cupper Points\n")
```

```
##   - Higher scores → higher predicted Cupper Points
```

```
cat("  - This aligns with domain knowledge about coffee quality\n")
```

```
##   - This aligns with domain knowledge about coffee quality
```

9.5.3 Comparison: PFI vs PDP-derived Importance

```
# Create comparison for features present in both analyses
comparison_methods <- pdp_variance_importance %>%
  left_join(
    pfi_results %>%
      filter(variable %in% pdp_variance_importance$Feature) %>%
      select(variable, Importance) %>%
      rename(Feature = variable, PFI_Importance = Importance),
    by = "Feature"
  ) %>%
  mutate(
    PFI_Rank = rank(-PFI_Importance),
    PDP_Rank = rank(-PDP_Variance)
  )

# Correlation of ranks
rank_correlation <- cor(comparison_methods$PFI_Rank, comparison_methods$PDP_Rank,
                        method = "spearman")

cat(sprintf("\nRank Correlation (Spearman) between PFI and PDP: %.3f\n", rank_correlation))
```

```
##
## Rank Correlation (Spearman) between PFI and PDP: -0.400
```

```r
# Plot comparison
comparison_long2 <- comparison_methods %>%
  select(Feature, PFI_Rank, PDP_Rank) %>%
  pivot_longer(cols = c(PFI_Rank, PDP_Rank),
               names_to = "Method", values_to = "Rank") %>%
  mutate(Method = ifelse(Method == "PFI_Rank", "PFI", "PDP Variance"))

ggplot(comparison_long2, aes(x = Feature, y = Rank, color = Method, group = Method)) +
  geom_point(size = 4) +
  geom_line(linewidth = 1) +
  scale_y_reverse() +  # Rank 1 at top
  scale_color_manual(values = c("PFI" = "#E63946", "PDP Variance" = "#457B9D")) +
  labs(title = "Feature Importance Rankings: PFI vs PDP",
       y = "Rank (1 = Most Important)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "bottom",
        plot.title = element_text(hjust = 0.5, face = "bold"))
```

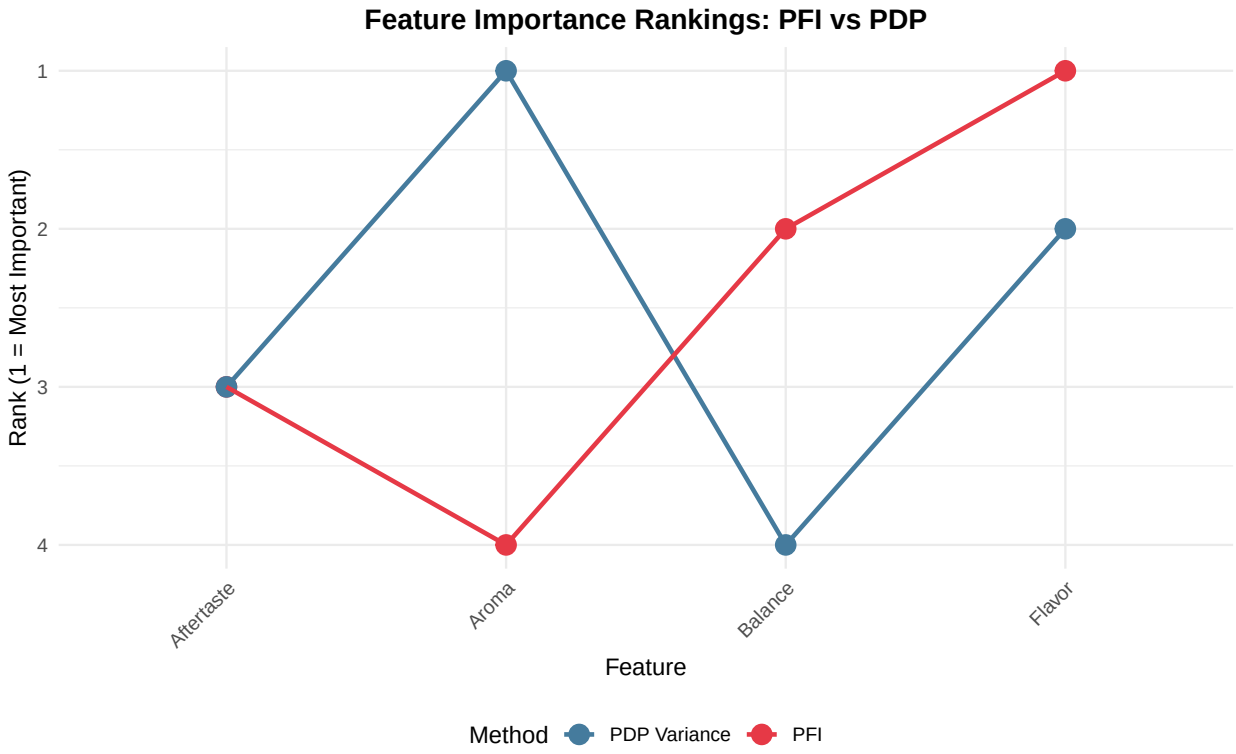


Figure 7: Comparison of feature importance rankings from PFI and PDP variance. Consistent rankings indicate robust importance estimates.

9.5.4 Practical Implications

```
cat("\nPRACTICAL IMPLICATIONS\n")
```

```
##
## PRACTICAL IMPLICATIONS
```

```
cat("======================\n\n")
```

```
## ======================
```

```
cat("1. MODEL TRUST:\n")
```

```
## 1. MODEL TRUST:
```

```
cat("   - PFI confirms the model uses sensible features (sensory scores)\n")
```

```
##    - PFI confirms the model uses sensible features (sensory scores)
```

```
cat("   - PDP shows expected positive relationships with quality indicators\n")
```

```
##    - PDP shows expected positive relationships with quality indicators
```

```
cat("   - These findings align with coffee quality expertise\n\n")
```

```
##    - These findings align with coffee quality expertise
```

```
cat("2. FEATURE ENGINEERING:\n")
```

```
## 2. FEATURE ENGINEERING:
```

```
cat("   - Focus data collection efforts on top PFI features\n")
```

```
##    - Focus data collection efforts on top PFI features
```

```
cat("   - Low-importance features could be removed for simpler models\n\n")
```

```
##    - Low-importance features could be removed for simpler models
```

```
cat("3. PREDICTION UNDERSTANDING:\n")
```

```
## 3. PREDICTION UNDERSTANDING:
```

```
cat("   - Break Down explains individual predictions\n")
```

```
##    - Break Down explains individual predictions
```

```
cat("   - Useful for auditing specific coffee quality assessments\n")
```

```
##    - Useful for auditing specific coffee quality assessments
```

```
cat("   - Identifies which scores drove high/low predictions\n")
```

```
##    - Identifies which scores drove high/low predictions
```

9.6 Accumulated Local Effects (ALE)

9.6.1 Concept

ALE plots show how features influence predictions by measuring the effect of small, local changes in feature values. Unlike PDP, ALE:

- **Handles correlated features correctly**: Evaluates effects along the actual data distribution
- **Shows local effects**: Accumulates changes in predictions within small intervals
- **Avoids extrapolation**: Only uses realistic feature combinations from the data

The ALE algorithm:

1. Divide feature range into intervals $[z_{k-1}, z_k]$
2. For each interval, calculate the average prediction difference when moving from z_{k-1} to z_k
3. Accumulate these differences: $\text{ALE}(x) = \sum_{k=1}^{k(x)} \Delta_k$
4. Center the curve so the mean effect is zero

9.6.2 Calculate ALE for Top Features

```
# Select top 6 features from PFI for ALE analysis
top_features <- head(pfi_results$variable, 6)

cat("Calculating ALE for top 6 features:\n")
```

```
## Calculating ALE for top 6 features:
```

```
cat(paste("  -", top_features, collapse = "\n"), "\n")
```

```
##   - Flavor
##   - Balance
##   - Aftertaste
##   - Aroma
##   - Body
##   - Acidity
```

```
# Calculate ALE for each top feature
set.seed(SEED)
ale_plots <- lapply(top_features, function(feature) {
  cat("  Processing:", feature, "...\n")
  model_profile(
    explainer = explainer_xgb_full,
    variables = feature,
    type = "accumulated",  # ALE method
    N = 1000               # Sample 1000 observations for efficiency
  )
})
```

```
##   Processing: Flavor ...
##   Processing: Balance ...
##   Processing: Aftertaste ...
```

```
##   Processing: Aroma ...
##   Processing: Body ...
##   Processing: Acidity ...
```

```r
names(ale_plots) <- top_features

# Manual correction so that ALE plots are centered around 0 which should be the default
ale_plots <- lapply(ale_plots, function(mp) {
  mp$agr_profiles$`_yhat_` <-
    mp$agr_profiles$`_yhat_` - mean(mp$agr_profiles$`_yhat_`)
  mp
})

cat("\nALE calculation complete.\n")
```

```
##
## ALE calculation complete.
```

9.6.3 ALE Visualizations

```r
# Create a grid of ALE plots
ale_plot_list <- lapply(seq_along(ale_plots), function(i) {
  feature <- names(ale_plots)[i]
  plot(ale_plots[[i]]) +
    ggtitle(paste0(i, ". ", feature)) +
    xlab(feature) +
    ylab("Accumulated Effect") +
    theme_minimal() +
    theme(
      plot.title = element_text(hjust = 0.5, size = 12, face = "bold"),
      axis.title = element_text(size = 10)
    )
})

# Arrange plots in a 2x3 grid
library(gridExtra)
do.call(grid.arrange, c(ale_plot_list, ncol = 2))
```

9.6.4 ALE Interpretation Guide

```r
cat("
**How to read ALE plots:**

- **Y-axis (Accumulated Effect)**: The impact on predicted Cupper Points
  - Values above 0: Feature values in this range increase predictions
  - Values below 0: Feature values in this range decrease predictions
  - The magnitude shows the strength of the effect
```

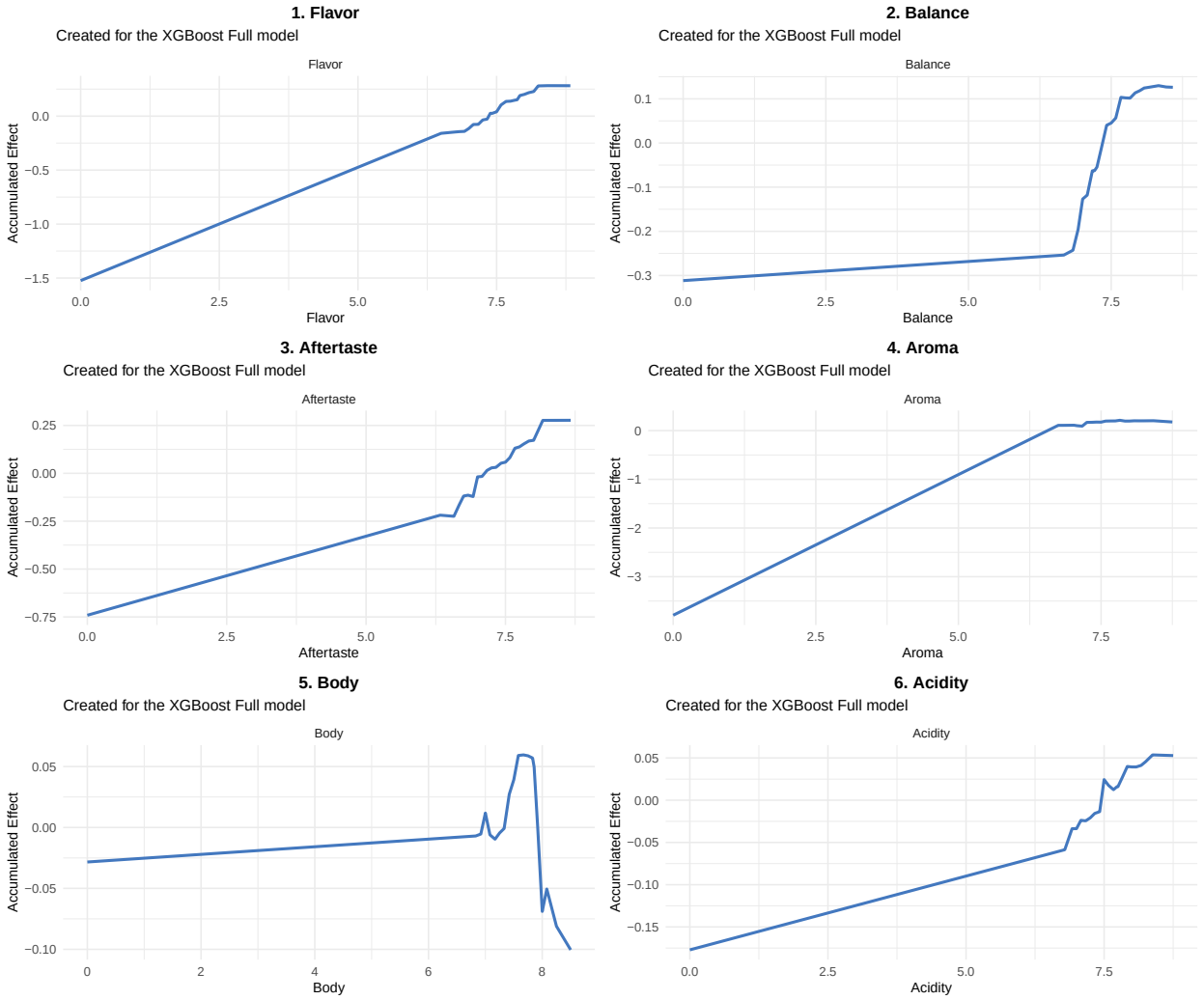


Figure 8: Accumulated Local Effects for the top 6 most important features. The y-axis shows the accumulated effect on predicted Cupper Points. Zero-centered curves indicate positive (above 0) or negative (below 0) influence on predictions.

```
- **X-axis**: The range of the feature's values in the training data

- **Curve shape**:
  - Steep slopes = strong local effects
  - Flat sections = feature has little effect in that range
  - Non-linear curves = complex, non-monotonic relationships

- **Zero-centering**: All curves are centered so their average effect is zero, making relative
")
```

How to read ALE plots:

- **Y-axis (Accumulated Effect)**: The impact on predicted Cupper Points
  - Values above 0: Feature values in this range increase predictions
  - Values below 0: Feature values in this range decrease predictions
  - The magnitude shows the strength of the effect
- **X-axis**: The range of the feature's values in the training data
- **Curve shape**:
  - Steep slopes = strong local effects
  - Flat sections = feature has little effect in that range
  - Non-linear curves = complex, non-monotonic relationships
- **Zero-centering**: All curves are centered so their average effect is zero, making relative comparisons easier

9.6.5 ALE Summary Statistics

```r
# Extract summary statistics for ALE plots
ale_summary <- data.frame(
  Feature = character(),
  Min_Effect = numeric(),
  Max_Effect = numeric(),
  Range = numeric(),
  Mean_Abs_Effect = numeric(),
  stringsAsFactors = FALSE
)

for (feature in names(ale_plots)) {
  ale_data <- ale_plots[[feature]]$agr_profiles
  effects <- ale_data$`_yhat_`

  ale_summary <- rbind(ale_summary, data.frame(
    Feature = feature,
    Min_Effect = min(effects),
    Max_Effect = max(effects),
    Range = max(effects) - min(effects),
```

```
    Mean_Abs_Effect = mean(abs(effects))
  ))
}

kable(ale_summary,
      col.names = c("Feature", "Min Effect", "Max Effect", "Effect Range", "Mean |Effect|"),
      caption = "ALE Effect Summary: Range and magnitude of accumulated effects for top feature
      format = "markdown", digits = 3)
```

Table 13: ALE Effect Summary: Range and magnitude of accumulated effects for top features

Feature	Min Effect	Max Effect	Effect Range	Mean |Effect|
Flavor	-1.524	0.282	1.806	0.196
Balance	-0.312	0.130	0.441	0.120
Aftertaste	-0.741	0.277	1.018	0.146
Aroma	-3.792	0.213	4.004	0.316
Body	-0.100	0.060	0.160	0.036
Acidity	-0.177	0.054	0.231	0.037

9.6.6 Key Insights from ALE

```
# Identify features with largest effect ranges
largest_range <- ale_summary[which.max(ale_summary$Range), ]
most_variable <- ale_summary[which.max(ale_summary$Mean_Abs_Effect), ]

cat(sprintf("
**Notable patterns:**

1. **Strongest overall effect**: %s (effect range: %.3f Cupper Points)
   - This feature shows the largest difference between its minimum and maximum accumulated effe

2. **Most variable effect**: %s (mean absolute effect: %.3f)
   - This feature shows the most consistent deviation from zero across its range

3. **Comparison with PFI**:
   - PFI tells us *which* features are important (breaking them hurts performance)
   - ALE tells us *how* they're important (the direction and magnitude of their effects)
   - Features can have high PFI but small ALE ranges if they're important for fine-tuning predi
",
largest_range$Feature, largest_range$Range,
most_variable$Feature, most_variable$Mean_Abs_Effect))
```

Notable patterns:

1. **Strongest overall effect**: Aroma (effect range: 4.004 Cupper Points)

    - This feature shows the largest difference between its minimum and maximum accumulated effects
2. **Most variable effect**: Aroma (mean absolute effect: 0.316)
    - This feature shows the most consistent deviation from zero across its range
3. **Comparison with PFI**:
    - PFI tells us *which* features are important (breaking them hurts performance)
    - ALE tells us *how* they're important (the direction and magnitude of their effects)
    - Features can have high PFI but small ALE ranges if they're important for fine-tuning predictions

9.7 Shapley Additive Explanations (SHAP)

9.7.1 Concept

SHAP values provide a unified measure of feature importance based on cooperative game theory. For each prediction, SHAP assigns each feature a value that represents its contribution to moving the prediction away from the baseline (average prediction).

Key properties of SHAP:

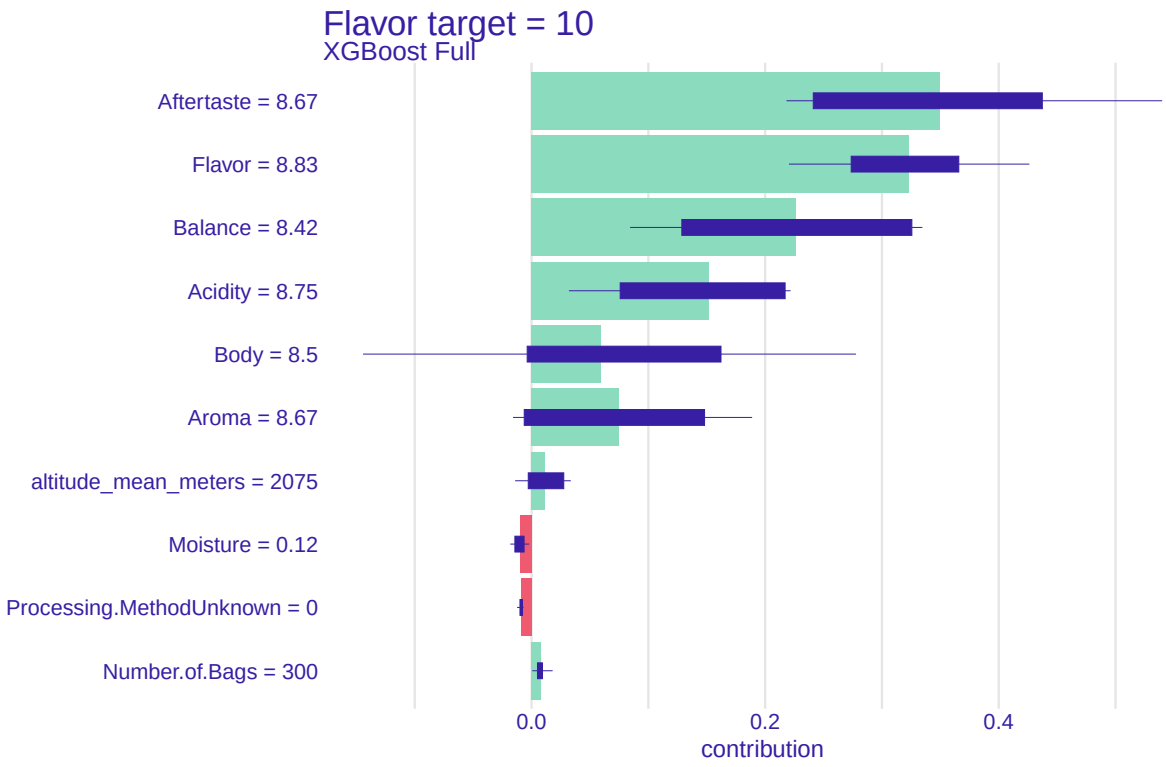
- **Local explanations**: Shows feature contributions for individual predictions
- **Global importance**: Aggregating SHAP values across observations reveals overall feature importance
- **Additive**: The sum of all SHAP values equals the difference between the prediction and the baseline
- **Fair attribution**: Based on Shapley values from game theory, ensuring fair credit assignment
- **Model-agnostic**: Works with any machine learning model

The intuition: SHAP answers "How much did each feature contribute to this specific prediction?"

```
set.seed(SEED)
# Find observation where flavor is closest to your target value - values 0.0, 10.0
target_val_flav <- 10.0

# Get the column index for flavor
flavor_col <- which(colnames(X_full_train) == "Flavor")

# Find observation closest to target value
obs_id <- which.min(abs(X_full_train[, flavor_col] - target_val_flav))

# Extract the observation as a numeric matrix row
new_obs_matrix <- X_full_train[obs_id, , drop = FALSE]

# Ensure it's numeric
storage.mode(new_obs_matrix) <- "numeric"

# Convert to data frame for predict_parts
new_obs <- as.data.frame(new_obs_matrix)

cat(paste0("Testing Observation #", obs_id, " where Flavor = ",
           round(X_full_train[obs_id, flavor_col], 2), "\n"))
```

```
## Testing Observation #661 where Flavor = 8.83
```

```
# Run SHAP for this observation
shap <- predict_parts(explainer = explainer_xgb_full,
                      new_observation = new_obs,
                      type = "shap", B = 25)
plot(shap) + ggtitle(paste("Flavor target =", target_val_flav))
```

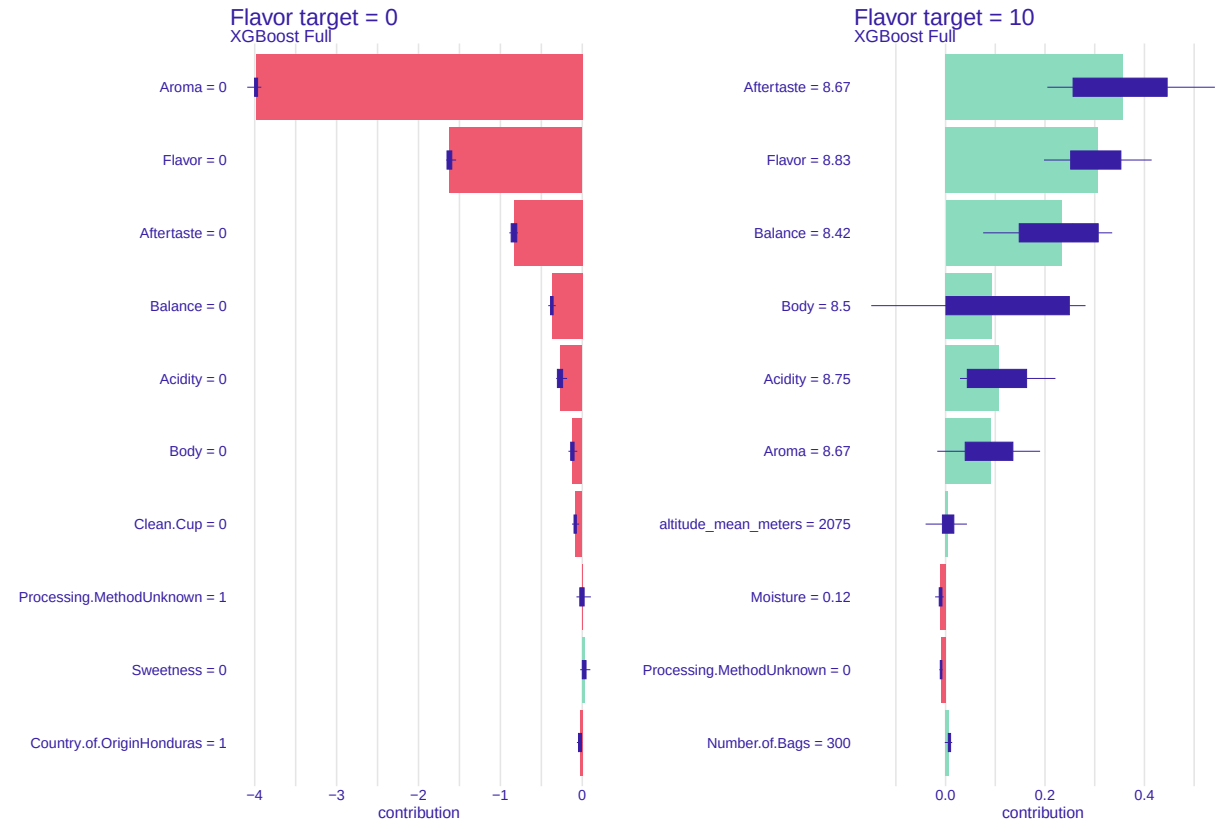



```r
set.seed(SEED)

# Target flavor values
target_vals <- c(0.0, 10.0)

# Get the column index for flavor
flavor_col <- which(colnames(X_full_train) == "Flavor")

shap_plots <- vector("list", length(target_vals))

for (i in seq_along(target_vals)) {

  target_val_flav <- target_vals[i]

  # Find observation closest to target value
  obs_id <- which.min(abs(X_full_train[, flavor_col] - target_val_flav))

  # Extract observation
  new_obs_matrix <- X_full_train[obs_id, , drop = FALSE]
  storage.mode(new_obs_matrix) <- "numeric"
  new_obs <- as.data.frame(new_obs_matrix)

  cat(paste0(
    "Testing Observation #", obs_id,
    " where Flavor = ",
```

```
    round(X_full_train[obs_id, flavor_col], 2), "\n"
  ))

  shap <- predict_parts(
    explainer = explainer_xgb_full,
    new_observation = new_obs,
    type = "shap",
    B = 25
  )

  shap_plots[[i]] <- plot(shap) +
    ggtitle(paste("Flavor target =", target_val_flav))
}
```

```
## Testing Observation #160 where Flavor = 0
## Testing Observation #661 where Flavor = 8.83
```

```
grid.arrange(
  grobs = shap_plots,
  ncol = 2
)
```

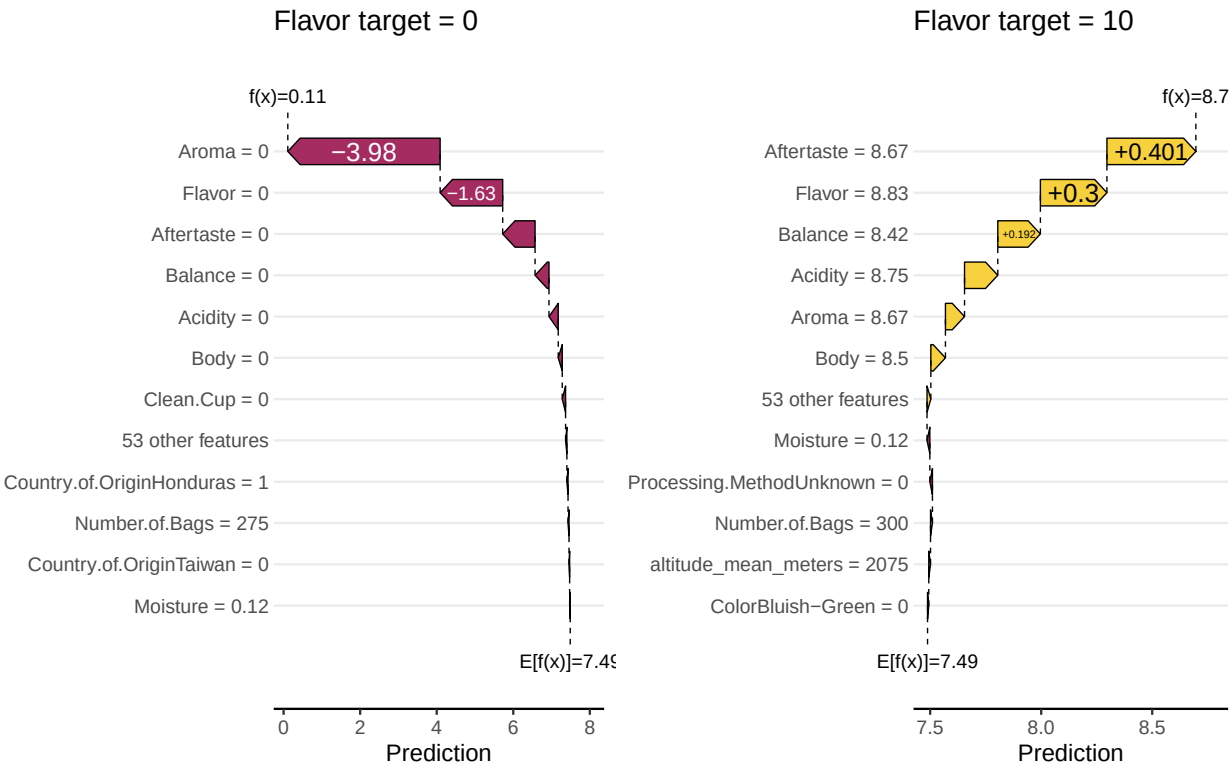



```
set.seed(SEED)
```

```r
# Target flavor values
target_vals <- c(0.0, 10.0)

# Column index for Flavor
flavor_col <- which(colnames(X_full_train) == "Flavor")

# Store plots
wf_plots <- vector("list", length(target_vals))

for (i in seq_along(target_vals)) {

  target_val_flav <- target_vals[i]

  # Find observation closest to target value
  obs_id <- which.min(abs(X_full_train[, flavor_col] - target_val_flav))

  # Extract observation
  new_obs_matrix <- X_full_train[obs_id, , drop = FALSE]
  storage.mode(new_obs_matrix) <- "numeric"
  #new_obs <- as.data.frame(new_obs_matrix)

  cat(paste0(
    "Testing Observation #", obs_id,
    " where Flavor = ",
    round(X_full_train[obs_id, flavor_col], 2), "\n"
  ))

  # Create shapviz object
  sv <- shapviz(
    explainer_xgb_full$model,
    X_pred = new_obs_matrix
  )

  # Waterfall plot
  wf_plots[[i]] <- sv_waterfall(sv, max_display = 12) +
    ggtitle(paste("Flavor target =", target_val_flav))
}
```

```
## Testing Observation #160 where Flavor = 0
## Testing Observation #661 where Flavor = 8.83
```

```r
# Arrange plots side-by-side
grid.arrange(
  grobs = wf_plots,
  ncol = 2
)
```


Flavor target = 0
f(x)=0.11
Aroma = 0
−3.98
Flavor = 0
−1.63
Aftertaste = 0
Balance = 0
Acidity = 0
Body = 0
Clean.Cup = 0
53 other features
Country.of.OriginHonduras = 1
Number.of.Bags = 275
Country.of.OriginTaiwan = 0
Moisture = 0.12
E[f(x)]=7.49
0
2
4
6
8
Prediction
Flavor target = 10
f(x)=8.7
Aftertaste = 8.67
+0.401
Flavor = 8.83
+0.3
Balance = 8.42
+0.192
Acidity = 8.75
Aroma = 8.67
Body = 8.5
53 other features
Moisture = 0.12
Processing.MethodUnknown = 0
Number.of.Bags = 300
altitude_mean_meters = 2075
ColorBluish−Green = 0
E[f(x)]=7.49
7.5
8.0
8.5
Prediction